Certificate-by-Resonance: ELPI Swap Search + F* NComm Kernel (MVP Spec)

Goal

Formalize an MVP **resonance-preserving proof/type engine** in a two-layer architecture:

1. **Trusted kernel (F*)** [2]
2. Defines a concrete semantic model for a tiny fragment.
3. Implements operators $A_{2,1}$, W_3, and projectors $\Pi_{p,1}$ for $p \in \{2, 3, 5\}$.
4. Defines **constructive noncommutation witnesses** `NComm` as counterexamples.
5. Verifies a witness by *computation* (exact rationals; no floats).
6. **Untrusted search (ELPI)** [1]
7. Searches over **operator-words** built from `{A21, W3, Pi(2), Pi(3), Pi(5)}`.
8. Uses a **resonance proxy** (order-sensitive) to propose local rewrites.
9. Applies **adjacent swaps** (local reordering) only if they improve resonance.
10. If a swap crosses the special noncommuting pair `(W3, A21)`, it must emit an **NComm certificate**; otherwise the swap is rejected.

This is the concrete "**certificate-by-resonance**" behavior:

- resonance drives *proposal/selection*
- certificates gate *acceptance*

Formal Objects

State space (fixed n)

- Fix $n = 30$.
- `State(30) := ℝ^{30}` represented as a total function `nat -> rat` (exact rationals).

Operators

Let `op` be the operator datatype:

- `A21` (parity mask) : zeroes odd indices.

- `W3` (mod-3 class averaging) : replaces each coordinate with the average over its residue class mod 3.
- `Pi(p)` for `p ∈ {2,3,5}` : class-averaging projectors at mod-`p` granularity.

Semantics:

- `apply : op -> state -> state`
- `eval_word : list op -> state -> state`

(Composition convention used in the MVP: `[o1;o2]` means `o2(o1(x))`.)

Noncommutation Witness Type (constructive)

Kernel witness object

A witness is **positive data**:

`ncomm_witness = { left: op; right: op; x_id: xid; idx: nat }`

Intended meaning:

- witness claims: `⟦left ∘ right⟧(x)[idx] ≠ ⟦right ∘ left⟧(x)[idx]`.

Kernel checker

`check_ncomm(w) : bool`

Computes both sides and checks inequality in exact rationals.

Canonical demo witness

A concrete separating counterexample:

- `left = W3`
- `right = A21`
- `x = δ1` (one-hot at index 1)
- `idx = 4`

Expected:

- `(W3 ∘ A21)(δ1) = 0`
- `(A21 ∘ W3)(δ1)[4] = 1/10`

So `check_ncomm(demo) = true`.

ELPI Search as Local Rewriting (Adjacent Swaps)

Word space

Words are lists of operators over the basis: `{A21, W3, Pi(2), Pi(3), Pi(5)}`

Resonance proxy (order-sensitive)

Define `score(word)`:

- Sum of base weights per operator.
- Plus adjacency bonuses `bonus(o_i, o_{i+1})`.
- Minus a small length penalty.

Order sensitivity ensures adjacent swaps can change score.

Local rewrite step

A single move swaps adjacent operators:

`swap_once : word -> word'` by exchanging one adjacent pair.

Improvement constraint

A swap is considered only if it strictly improves resonance:

`Gain = score(word') - score(word) > 0`.

Certificate gate for order-sensitive swaps

If a proposed swap is between `W3` and `A21` (either orientation), the move is only allowed if it emits an `NComm` certificate.

In the MVP, the certificate is represented as a basis generator:

- `ncomm_w3_a21` with payload `(Delta1, idx=4)`.

Acceptance rule (conceptual):

- If swapping crosses `(W3, A21)`, require `cert_ok(ncomm_w3_a21)`.
- Otherwise reject the swap.

`cert_ok` can be:

- **Demo mode:** a local fact in ELPI (assume validated)
- **Real mode:** a system call into an F*-compiled checker executable returning exit code 0.

Drift basis

Drift is tracked as a **set-like basis** (no duplicates):

- `DriftBasis := list witness` with insertion `insert_w`.

Drift evolves **only when a gated swap is actually used**.

Algorithm (Hill-Climb with Certified Swaps)

Inputs:

- initial word `W0`
- initial drift `D0 = []`
- step budget `K`

Loop:

1. Enumerate all single adjacent swaps of the current word.
2. Filter to those with `Gain > 0`.
3. For each candidate, apply certificate gate:
4. if swap is between `W3` and `A21`, require `NComm` cert.
5. Choose the candidate with maximal gain.
6. Update word and drift basis.
7. Stop if no improving certified swap exists, or budget exhausted.

Outputs:

- final word `W*`
- final score
- final drift basis `D*`

Invariants and Extensions

Soundness boundary

- **Kernel** is the only authority on witness validity.
- **Resonance** is untrusted and cannot affect acceptance except via proposing rewrites.

Next minimal extension (toward the blueprint)

Upgrade `ncomm_w3_a21` to a parameterized witness:

`ncomm(left,right,xid,idx)`

and pass those parameters to `kernel_check` so ELPI can validate **newly discovered** witnesses, not just a fixed demo.

Deliverables

1. `Kernel.fst`
2. exact rational arithmetic
3. operators `A21`, `W3`, `Pi(2/3/5)`
4. `NComm` witness type + checker
5. `demo_ok : bool` exposed for integration
6. `rewrite_swap.elpi`
7. basis, score proxy, adjacency swap generator
8. certified hill-climb
9. drift basis insertion and trace output
10. Optional integration
11. build `kernel_check` executable from F* codegen (OCaml)
12. switch `cert_ok` in ELPI from demo fact to `std.system` call

Acceptance Criteria (MVP)

- F* verifies `demo_ok = true` by computation.
- ELPI performs at least one improving swap.
- If an improving swap crosses `(W3, A21)`, ELPI only accepts it while emitting the `NComm` certificate.
- Drift basis remains empty if no certified gated swap is used; becomes `{ncomm_w3_a21}` precisely when such a swap is used.

Notes

- This MVP is intentionally **semantic-first**: the noncommutation witness is a countermodel, so it is constructive and checkable.

- This matches the intended design: **order effects are proof-relevant data** and are tracked as a compact drift basis.

References

[1] C. Dunchev, F. Guidi, C. Sacerdoti Coen, E. Tassi. *ELPI: Fast, Embeddable, λProlog Interpreter.* In: **LPAR-20** (Logic for Programming, Artificial Intelligence and Reasoning), LNCS. 2015. DOI: 10.1007/978-3-662-48899-7_32.

[2] N. Swamy et al. *Dependent Types and Multi-Monadic Effects in F*.* In: **POPL 2016** (Proceedings of the 43rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages). 2016. DOI: 10.1145/2914770.2837655.

[3] R. Harper, F. Honsell, G. Plotkin. *A Framework for Defining Logics.* **Journal of the ACM** 40(1). 1993. DOI: 10.1145/138027.138060.

[4] F. Pfenning, C. Schürmann. *System Description: Twelf — A Meta-Logical Framework for Deductive Systems.* **CADE-16 / Computation and Deduction**. 1999. (ACM entry: 10.5555/648235.753634).

[5] B. Pientka, J. Dunfield. *Beluga: A Framework for Programming and Reasoning with Deductive Systems (System Description).* In: **IJCAR 2010**, LNCS 6173. 2010. DOI: 10.1007/978-3-642-14203-1_2.

[6] TypeDB (Vaticle). Project site and documentation: *TypeDB* (database and TypeQL ecosystem). See typedb.com and the TypeDB open-source repository.

[7] C. Dorn et al. *TypeQL: A Type-Theoretic & Polymorphic Query Language.* **ACM**. 2024. DOI: 10.1145/3651611.